

Responsive Design & State Management

Frontend Development

Mobile-First Design in a Component World

Building responsively inside a component framework works the same way as plain CSS — mobile-first styles, layered breakpoints — but with an added question: should a component change its layout with CSS alone, or actually render different content at different sizes? As a rule, prefer CSS-only responsiveness (hiding/showing elements, changing grid layouts) wherever possible, since it's simpler and doesn't cause a re-render; only conditionally render entirely different components when the mobile and desktop experiences are genuinely structurally different, not just re-styled.

Cross-Browser Testing

Different browsers (and different versions of the same browser) can render CSS and execute JavaScript slightly differently. Practical cross-browser testing means: checking layout in at least one Chromium-based browser, Firefox, and Safari (which has historically lagged on newer CSS/JS features); testing on a real mobile device or accurate emulator, not just a resized desktop browser window, since touch interactions and viewport behavior differ; and treating browser dev tools' device toolbar as a first approximation, not a substitute for testing on real hardware before shipping something widely used.

Global vs. Local State, Revisited

A common mistake is putting too much state globally, which makes an app harder to reason about since any component could theoretically change shared data. A practical guideline: start with local state in the component that needs it; only lift it up to a shared/global store once a second, unrelated component genuinely needs to read or change the same value. Global state should represent things that are truly cross-cutting — the logged-in user, theme preference, shopping cart — not values only one screen cares about.

Performance Basics

The most common frontend performance issues come from unnecessary re-renders and unnecessarily large bundles.

- Avoid creating new objects/functions inline in render if they're passed to child components that otherwise wouldn't re-render — this can defeat memoization.
- Use code-splitting/lazy-loading (loading a route or component's code only when it's actually needed) to keep the initial page load small.
- Optimize images (correct format, compressed, appropriately sized) — images are usually

the single largest contributor to page weight.

- Measure before optimizing: browser dev tools' Performance and Network tabs (or Lighthouse) tell you what's actually slow, rather than guessing.

Building a Capstone Responsive Application

Bringing the module together: a well-built responsive application with proper state management starts from a clear component hierarchy planned before writing code, uses local state by default and lifts state up only when genuinely shared, applies mobile-first CSS with a small number of deliberate breakpoints, and is verified on at least one real mobile device before being considered done. Treating responsiveness and state architecture as decisions made upfront — not patched in after the fact — is what separates a capstone project that holds together from one that needs a rewrite the moment new requirements arrive.

Learn More

web.dev — Learn Responsive Design

<https://web.dev/learn/design/>

web.dev — Web Performance

<https://web.dev/learn/performance/>

MDN Web Docs — Cross Browser Testing

https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Testing